Modern Digital Systems Design Course No: ENCE4110

Laboratory Report

UART-Based Case-Insensitive HELLO Detector Using a Moore FSM

Submitted By: Monowar Mahmud
DU ID: 873827421
Date: October 24, 2026

Objective

Design and implement a modular Verilog system that receives ASCII characters from a laptop through UART, detects the sequence `HELLO` without regard to letter case, and blinks the detected word across five seven-segment displays for three seconds. During ordinary operation, the most recently received character is displayed on HEX0 and its ASCII value is shown on LEDR[7:0].

System Overview

The design targets the MAX 10 FPGA development board using the 50 MHz `MAX10_CLK1_50` clock. The top-level `main` module connects the UART receiver, UART transmitter (echo), sequence detector, timing counter, and five ASCII-to-seven-segment decoders. UART data is processed only when `RxD_data_ready` is asserted.

Moore FSM Design

The FSM uses six states, S_0 through S_5. States S_1–S_4 represent successful recognition of successive prefixes of `hello`; S_5 is the detected/blinking state. The uppercase ASCII range 65–90 is normalized to lowercase by adding 32. State transitions are evaluated for new received bytes only (`data_valid`=1), except the completion transition from S_5.

State	Matched prefix / function	Expected input or exit
S_0	Idle	h $\rightarrow S_1$
S_1	h	e $\rightarrow S_2$
S_2	he	l $\rightarrow S_3$
S_3	hel	l $\rightarrow S_4$
S_4	hell	o $\rightarrow S_5$
S_5	HELLO detected, blinking	counter_done $\rightarrow S_0$

For mismatch inputs in S_1–S_4, the FSM returns to S_0, except a new `h` returns to S_1. The outputs `hello_detected` and `counter_en` are asserted only in S_5, making them Moore outputs dependent on the registered state.

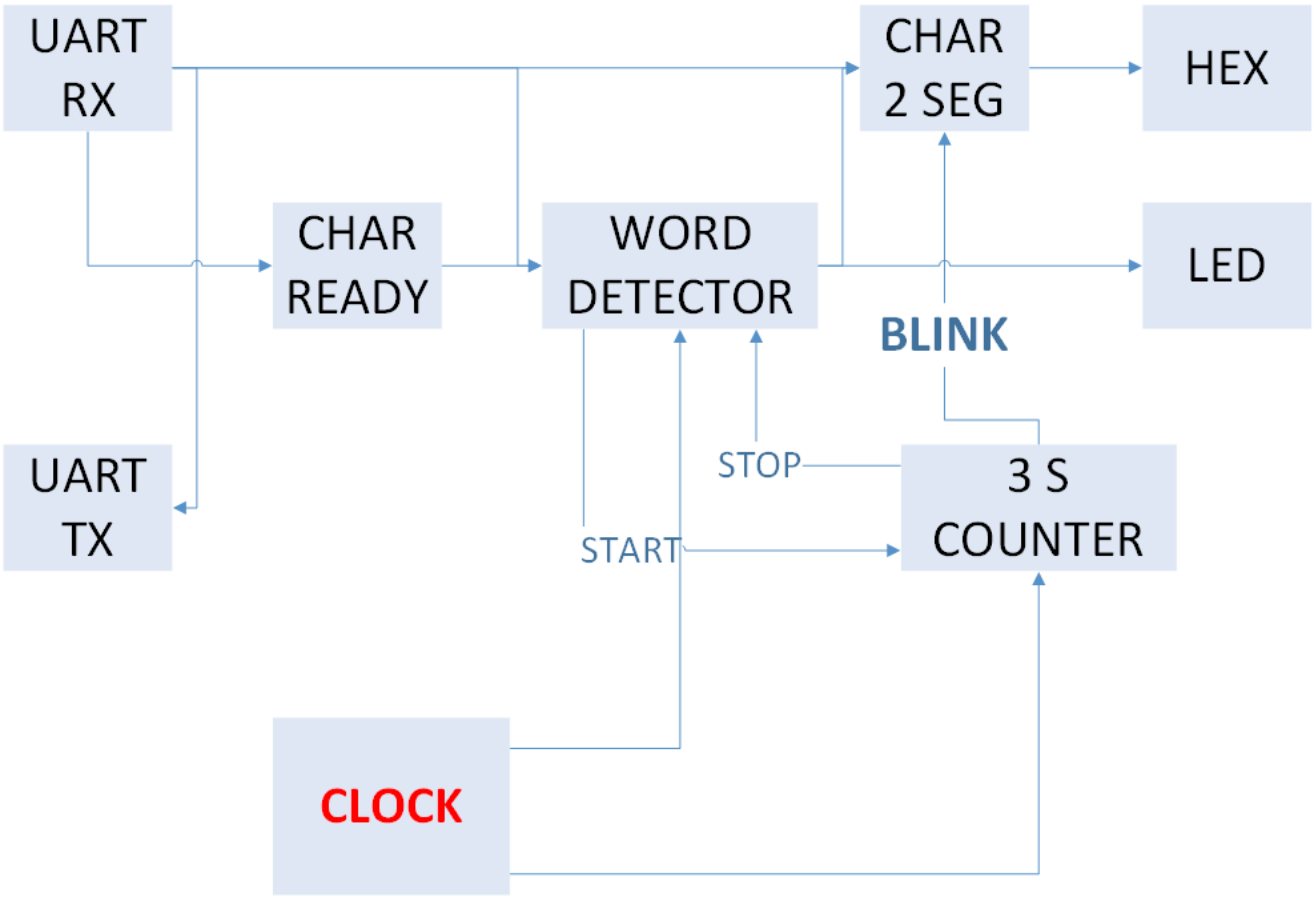


Figure 1: High-level block diagram of the UART-based HELLO detection system.

Counter and Display Operation

The 50 MHz clock yields

$$N_{\mathrm{ON/OFF}} = 50 \times 10^6(0.25) = 12{,}500{,}000,$$
$$N_{\mathrm{total}} = 50 \times 10^6(3) = 150{,}000{,}000.$$

The `blink_counter` contains a 24-bit blink interval counter and a 28-bit total-duration counter. While enabled, `blink` toggles every 0.25 s, creating a 0.5 s complete ON/OFF period. At the three-second threshold, `counter_done` triggers the FSM return to idle. Both counters reset when disabled.

In ordinary operation, HEX0 decodes `GPout`, HEX1–HEX4 are blank, and LEDR[7:0] shows the latest UART byte. In S_5, the five displays show `HELLO` during the ON interval and blank during the OFF interval. LEDR[8] indicates detection and LEDR[9] follows the blinking indication. The UART transmitter echoes received bytes.

RTL Synthesis Views

The following figures are the provided Quartus RTL Viewer screenshots, retained as `image_1` through `image_3`.

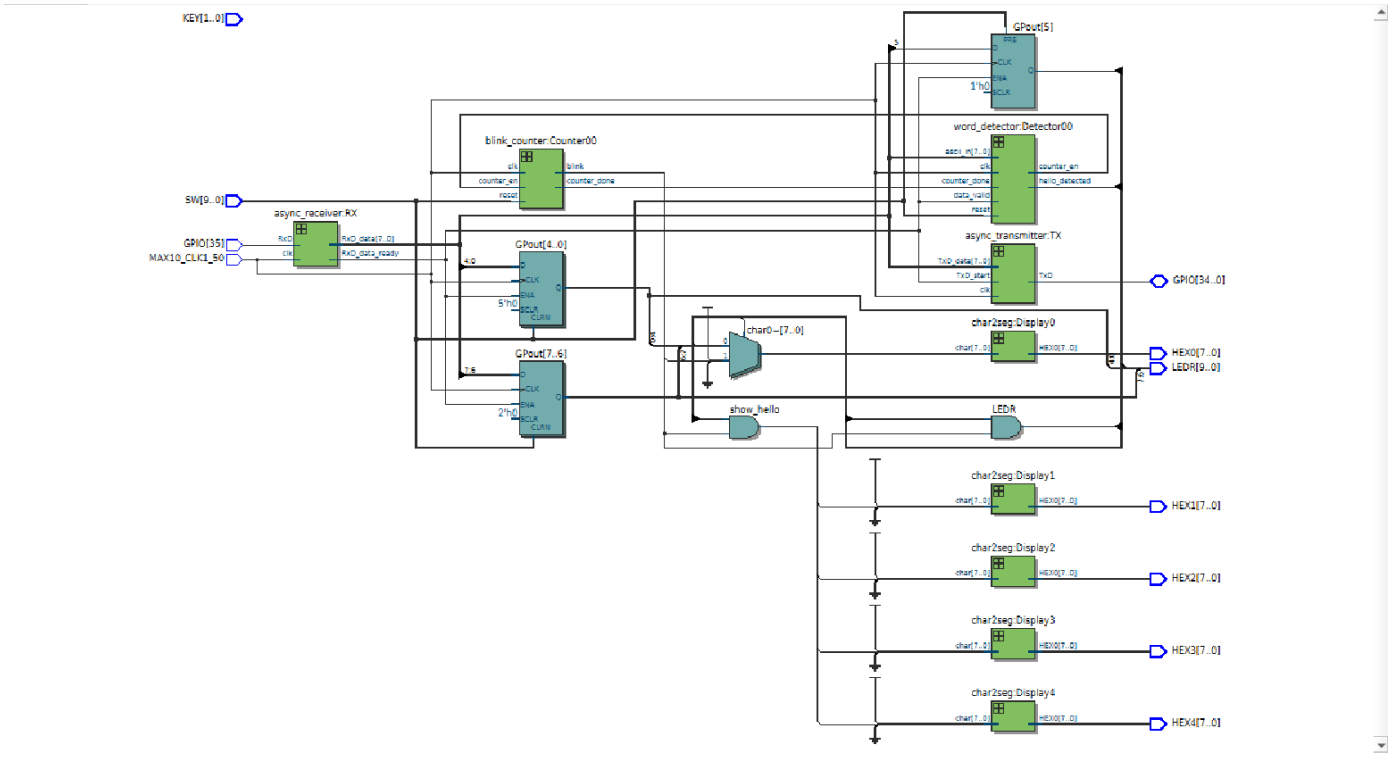


Figure 2: RTL view of the top-level system and module interconnections (image_1).

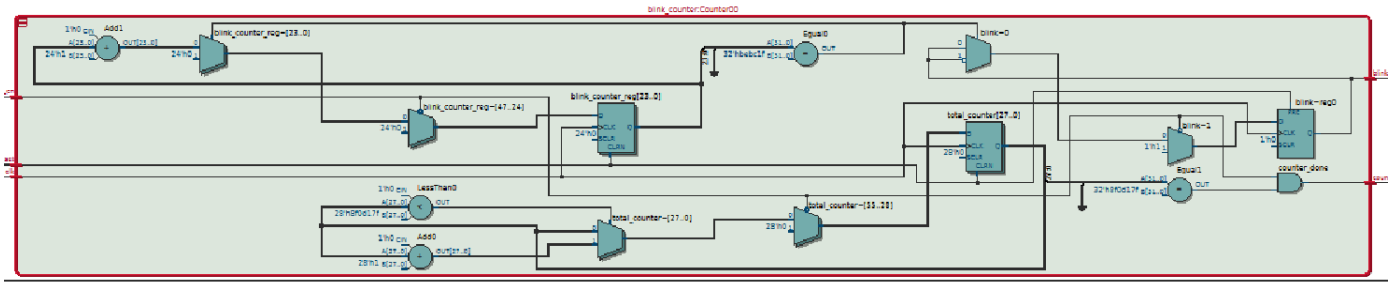


Figure 3: Expanded RTL implementation of the blink and duration counter (image_2).

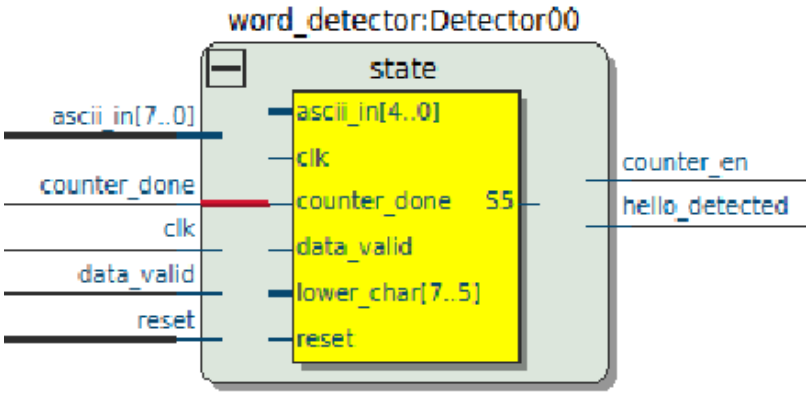


Figure 4: RTL view of the word-detector FSM (image_3).

Functional Verification Plan

Input / condition	Expected behavior
`H`, `e`, `L`, `l`, `o`	Case-insensitive match; enter S_5 on final `o`.
`HELXO`	No HELLO detection; latest character remains on HEX0.
Nonmatching byte	Show received byte on HEX0 and LEDR[7:0].
After HELLO match	HEX4–HEX0 blink word with 0.25 s ON and 0.25 s OFF for 3 s.
Counter completion	Return to idle and restore ordinary HEX0 display.
Reset asserted	Return FSM to S_0 and clear timing registers.

These are expected outcomes derived from the attached RTL source; no simulation waveforms or measured board results were supplied for independent verification. Bytes received while S_5 is active are echoed and stored in `GPout`, but ignored by the word detector. Note also that the decoder intentionally maps uppercase letters to lowercase segment patterns.

Conclusion

The modular design combines serial ASCII reception, a six-state Moore sequence detector, a reusable timing counter, and five seven-segment decoders. The architecture detects `HELLO` in any mixture

of letter cases, maintains ordinary character display outside the detection state, and controls a three-second blinking response. The supplied RTL Viewer figures document the synthesized module hierarchy and internal counter and FSM structures.

Appendix: Verilog Source Listings

The following listings reproduce the supplied implementation files. The UART receiver source was not included in the latest attachments, although its module appears in the top-level RTL.

A. Top-Level Module (main.v)

```
‘default_nettype none

module main (
    input wire MAX10_CLK1_50,
    input wire [9:0] SW,
    input wire [1:0] KEY,
    output wire [9:0] LEDR,
    inout wire [35:0] GPIO,

    output wire [7:0] HEX0,
    output wire [7:0] HEX1,
    output wire [7:0] HEX2,
    output wire [7:0] HEX3,
    output wire [7:0] HEX4
);

    wire w_clk = MAX10_CLK1_50;
    wire w_reset = SW[9];

    wire RxD_data_ready;
    wire [7:0] RxD_data;
    reg [7:0] GPout;

    wire counter_in;
    wire counter_out;
    wire blink;
    wire hello_detected;

    wire show_hello;

    // UART receiver
    async_receiver RX (
        .clk(w_clk),
        .RxD(GPIO[35]),
        .RxD_data_ready(RxD_data_ready),
        .RxD_data(RxD_data)
    );

    // Store latest UART character
    always @(posedge w_clk or posedge w_reset) begin
        if (w_reset)
            GPout <= 8’h20;
        else if (RxD_data_ready)
            GPout <= RxD_data;
    end

    // UART transmitter (echo)
    async_transmitter TX (
        .clk(w_clk),
        .TxD(GPIO[33]),
        .TxD_start(RxD_data_ready),
        .TxD_data(RxD_data)
    );

    // HELLO sequence detector
    word_detector Detector00 (
        .clk(w_clk),
        .reset(w_reset),
        .ascii_in(RxD_data),
        .data_valid(RxD_data_ready),
        .counter_done(counter_out),
```

```
        .counter_en(counter_in),
        .hello_detected(hello_detected)
    );

    // Existing counter: unchanged
    blink_counter Counter00 (
        .clk(w_clk),
        .reset(w_reset),
        .counter_en(counter_in),
        .counter_done(counter_out),
        .blink(blink)
    );

    // Enable HELLO display during ON interval
    assign show_hello = hello_detected && blink;

    // Normal display or HELLO display
    wire [7:0] char0;
    wire [7:0] char1;
    wire [7:0] char2;
    wire [7:0] char3;
    wire [7:0] char4;

    assign char0 = hello_detected ?
                   (blink ? "o" : " ") : GPout;

    assign char1 = show_hello ? "l" : " ";
    assign char2 = show_hello ? "l" : " ";
    assign char3 = show_hello ? "e" : " ";
    assign char4 = show_hello ? "h" : " ";

    // Seven-segment decoders
    char2seg Display0 (.char(char0), .HEX0(HEX0));
    char2seg Display1 (.char(char1), .HEX0(HEX1));
    char2seg Display2 (.char(char2), .HEX0(HEX2));
    char2seg Display3 (.char(char3), .HEX0(HEX3));
    char2seg Display4 (.char(char4), .HEX0(HEX4));

    // LEDs show latest UART ASCII value
    assign LEDR[7:0] = GPout;
    assign LEDR[8] = hello_detected;
    assign LEDR[9] = blink && hello_detected;

endmodule

`default_nettype wire
```

B. Word Detector (word_detector.v)

```
`default_nettype none

module word_detector (
    input wire clk,
    input wire reset,
    input wire [7:0] ascii_in,
    input wire data_valid,
    input wire counter_done,

    output wire counter_en,
    output wire hello_detected
);

    localparam S0 = 3'd0; // Idle
    localparam S1 = 3'd1; // H
    localparam S2 = 3'd2; // HE
    localparam S3 = 3'd3; // HEL
    localparam S4 = 3'd4; // HELL
    localparam S5 = 3'd5; // HELLO detected
```

```
reg [2:0] state, next_state;

wire [7:0] lower_char;

// Convert uppercase to lowercase
assign lower_char =
    (ascii_in >= 8'd65 && ascii_in <= 8'd90)
    ? ascii_in + 8'd32 : ascii_in;

// State register
always @(posedge clk or posedge reset) begin
    if (reset)
        state <= S0;
    else
        state <= next_state;
end

// Next-state logic
always @(*) begin
    next_state = state;

    case (state)
        S0: if (data_valid) begin
            if (lower_char == "h")
                next_state = S1;
        end

        S1: if (data_valid) begin
            if (lower_char == "e")
                next_state = S2;
            else if (lower_char == "h")
                next_state = S1;
            else
                next_state = S0;
        end

        S2: if (data_valid) begin
            if (lower_char == "l")
                next_state = S3;
            else if (lower_char == "h")
                next_state = S1;
            else
                next_state = S0;
        end

        S3: if (data_valid) begin
            if (lower_char == "l")
                next_state = S4;
            else if (lower_char == "h")
                next_state = S1;
            else
                next_state = S0;
        end

        S4: if (data_valid) begin
            if (lower_char == "o")
                next_state = S5;
            else if (lower_char == "h")
                next_state = S1;
            else
                next_state = S0;
        end

        S5: begin
            if (counter_done)
                next_state = S0;
        end

        default: next_state = S0;
    endcase
end

// Moore outputs
```

```
    assign hello_detected = (state == S5);
    assign counter_en = (state == S5);

endmodule

`default_nettype wire
```

C. Blink Counter (blink_counter.v)

```
`default_nettype none

module blink_counter (
    input wire clk,
    input wire reset,
    input wire counter_en,

    output wire counter_done,
    output reg blink
);

    parameter CLK_FREQ = 50000000;

    localparam TOTAL_COUNT = 3 * CLK_FREQ;
    localparam BLINK_COUNT = CLK_FREQ / 4;

    reg [27:0] total_counter;
    reg [23:0] blink_counter_reg;

    // Total 3-second counter
    always @(posedge clk or posedge reset) begin
        if (reset)
            total_counter <= 28'd0;

        else if (!counter_en)
            total_counter <= 28'd0;

        else if (total_counter < TOTAL_COUNT - 1)
            total_counter <= total_counter + 1'b1;
    end

    // 0.25-second blink counter
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            blink_counter_reg <= 24'd0;
            blink <= 1'b1;
        end

        else if (!counter_en) begin
            blink_counter_reg <= 24'd0;
            blink <= 1'b1;
        end

        else if (blink_counter_reg == BLINK_COUNT - 1) begin
            blink_counter_reg <= 24'd0;
            blink <= ~blink;
        end

        else
            blink_counter_reg <= blink_counter_reg + 1'b1;
    end

    // Completion signal
    assign counter_done =
        counter_en && (total_counter == TOTAL_COUNT - 1);

endmodule

`default_nettype wire
```

D. Seven-Segment Decoder (char2seg.v)

```
module char2seg(
        input           [7:0] char,
        output reg      [7:0] HEX0
);

        wire [7:0] w_char;

        assign w_char = ((char >= 8'd65) && (char <= 8'd90)) ? (char + 8'd32 ) : char;

        always @(*) begin

                HEX0 = 8'hFF;

                // ASCII Table Mapping
                case(w_char)
                        // Numbers
                        8'd48: HEX0 = 8'b01000000; // 0
                        8'd49: HEX0 = 8'b01111001; // 1
                        8'd50: HEX0 = 8'b00100100; // 2
                        8'd51: HEX0 = 8'b00110000; // 3
                        8'd52: HEX0 = 8'b00011001; // 4
                        8'd53: HEX0 = 8'b00010010; // 5
                        8'd54: HEX0 = 8'b00000010; // 6
                        8'd55: HEX0 = 8'b01111000; // 7
                        8'd56: HEX0 = 8'b00000000; // 8
                        8'd57: HEX0 = 8'b00011000; // 9
                        // Letters
                        8'd97: HEX0 = 8'b10001000; // a
                        8'd98: HEX0 = 8'b10000011; // b
                        8'd99: HEX0 = 8'b11000110; // c
                        8'd100: HEX0 = 8'b10100001; // d
                        8'd101: HEX0 = 8'b10000110; // E
                        8'd102: HEX0 = 8'b10001110; // F
                        8'd103: HEX0 = 8'b11000010; // G
                        8'd104: HEX0 = 8'b10001001; // H
                        8'd105: HEX0 = 8'b11111001; // I
                        8'd106: HEX0 = 8'b11110001; // J
                        8'd107: HEX0 = 8'b10011100; // k?
                        8'd108: HEX0 = 8'b11000111; // L
                        8'd109: HEX0 = 8'b10011100; // M?
                        8'd110: HEX0 = 8'b10101011; // n
                        8'd111: HEX0 = 8'b10100011; // o
                        8'd112: HEX0 = 8'b10001100; // p
                        8'd113: HEX0 = 8'b10011000; // q
                        8'd114: HEX0 = 8'b10101111; // r
                        8'd115: HEX0 = 8'b10010010; // s
                        8'd116: HEX0 = 8'b11111000; // t
                        8'd117: HEX0 = 8'b11100011; // u
                        8'd118: HEX0 = 8'b10011100; // v?
                        8'd119: HEX0 = 8'b10011100; // w?
                        8'd120: HEX0 = 8'b10011100; // x?
                        8'd121: HEX0 = 8'b10011100; // y?
                        8'd122: HEX0 = 8'b10100100; // z
                endcase
        end

endmodule
```

E. UART Transmitter (async_transmitter.v)

```
////////////////////////////////////////////////////
// RS-232 RX and TX module
// (c) fpga4fun.com & KNJN LLC - 2003 to 2016

// The RS-232 settings are fixed
// TX: 8-bit data, 2 stop, no-parity
// RX: 8-bit data, 1 stop, no-parity (the receiver can accept more stop bits of course)
```

```
//`define SIMULATION   // in this mode, TX outputs one bit per clock cycle
                       // and RX receives one bit per clock cycle (for fast simulations)

////////////////////////////////////////////////////
module async_transmitter(
        input clk,
        input TxD_start,
        input [7:0] TxD_data,
        output TxD,
        output TxD_busy
);

// Assert TxD_start for (at least) one clock cycle to start transmission of TxD_data
// TxD_data is latched so that it doesn't have to stay valid while it is being sent

parameter ClkFrequency = 50000000;      // 50MHz
parameter Baud = 115200;

generate
        if(ClkFrequency<Baud*8 && (ClkFrequency % Baud!=0)) ASSERTION_ERROR PARAMETER_OUT_OF_RANGE("Frequency incompatible
      with requested Baud rate");
endgenerate

////////////////////////////////
`ifdef SIMULATION
wire BitTick = 1'b1;  // output one bit per clock cycle
`else
wire BitTick;
BaudTickGen #(ClkFrequency, Baud) tickgen(.clk(clk), .enable(TxD_busy), .tick(BitTick));
`endif

reg [3:0] TxD_state = 0;
wire TxD_ready = (TxD_state==0);
assign TxD_busy = ~TxD_ready;

reg [7:0] TxD_shift = 0;
always @(posedge clk)
begin
        if(TxD_ready & TxD_start)
                TxD_shift <= TxD_data;
        else
        if(TxD_state[3] & BitTick)
                TxD_shift <= (TxD_shift >> 1);

        case(TxD_state)
                4'b0000: if(TxD_start) TxD_state <= 4'b0100;
                4'b0100: if(BitTick) TxD_state <= 4'b1000;  // start bit
                4'b1000: if(BitTick) TxD_state <= 4'b1001;  // bit 0
                4'b1001: if(BitTick) TxD_state <= 4'b1010;  // bit 1
                4'b1010: if(BitTick) TxD_state <= 4'b1011;  // bit 2
                4'b1011: if(BitTick) TxD_state <= 4'b1100;  // bit 3
                4'b1100: if(BitTick) TxD_state <= 4'b1101;  // bit 4
                4'b1101: if(BitTick) TxD_state <= 4'b1110;  // bit 5
                4'b1110: if(BitTick) TxD_state <= 4'b1111;  // bit 6
                4'b1111: if(BitTick) TxD_state <= 4'b0010;  // bit 7
                4'b0010: if(BitTick) TxD_state <= 4'b0011;  // stop1
                4'b0011: if(BitTick) TxD_state <= 4'b0000;  // stop2
                default: if(BitTick) TxD_state <= 4'b0000;
        endcase
end

assign TxD = (TxD_state<4) | (TxD_state[3] & TxD_shift[0]);  // put together the start, data and stop bits
endmodule


////////////////////////////////////////////////////
module async_receiver(
        input clk,
        input RxD,
        output reg RxD_data_ready = 0,
        output reg [7:0] RxD_data = 0,  // data received, valid only (for one clock cycle) when RxD_data_ready is asserted
```

```
        // We also detect if a gap occurs in the received stream of characters
        // That can be useful if multiple characters are sent in burst
        //  so that multiple characters can be treated as a "packet"
        output RxD_idle,  // asserted when no data has been received for a while
        output reg RxD_endofpacket = 0  // asserted for one clock cycle when a packet has been detected (i.e. RxD_idle is
    going high)
);

parameter ClkFrequency = 50000000; // 25MHz
parameter Baud = 115200;

parameter Oversampling = 8;  // needs to be a power of 2
// we oversample the RxD line at a fixed rate to capture each RxD data bit at the "right" time
// 8 times oversampling by default, use 16 for higher quality reception

generate
        if(ClkFrequency<Baud*Oversampling) ASSERTION_ERROR PARAMETER_OUT_OF_RANGE("Frequency too low for current Baud rate
     and oversampling");
        if(Oversampling<8 || ((Oversampling & (Oversampling-1))!=0)) ASSERTION_ERROR PARAMETER_OUT_OF_RANGE("Invalid
    oversampling value");
endgenerate

////////////////////////////////
reg [3:0] RxD_state = 0;

`ifdef SIMULATION
wire RxD_bit = RxD;
wire sampleNow = 1'b1;  // receive one bit per clock cycle

`else
wire OversamplingTick;
BaudTickGen #(ClkFrequency, Baud, Oversampling) tickgen(.clk(clk), .enable(1'b1), .tick(OversamplingTick));

// synchronize RxD to our clk domain
reg [1:0] RxD_sync = 2'b11;
always @(posedge clk) if(OversamplingTick) RxD_sync <= {RxD_sync[0], RxD};

// and filter it
reg [1:0] Filter_cnt = 2'b11;
reg RxD_bit = 1'b1;

always @(posedge clk)
if(OversamplingTick)
begin
        if(RxD_sync[1]==1'b1 && Filter_cnt!=2'b11) Filter_cnt <= Filter_cnt + 1'd1;
        else
        if(RxD_sync[1]==1'b0 && Filter_cnt!=2'b00) Filter_cnt <= Filter_cnt - 1'd1;

        if(Filter_cnt==2'b11) RxD_bit <= 1'b1;
        else
        if(Filter_cnt==2'b00) RxD_bit <= 1'b0;
end

// and decide when is the good time to sample the RxD line
function integer log2(input integer v); begin log2=0; while(v>>log2) log2=log2+1; end endfunction
localparam l2o = log2(Oversampling);
reg [l2o-2:0] OversamplingCnt = 0;
always @(posedge clk) if(OversamplingTick) OversamplingCnt <= (RxD_state==0) ? 1'd0 : OversamplingCnt + 1'd1;
wire sampleNow = OversamplingTick && (OversamplingCnt==Oversampling/2-1);
`endif

// now we can accumulate the RxD bits in a shift-register
always @(posedge clk)
case(RxD_state)
        4'b0000: if(~RxD_bit) RxD_state <= `ifdef SIMULATION 4'b1000 `else 4'b0001 `endif;  // start bit found?
        4'b0001: if(sampleNow) RxD_state <= 4'b1000;  // sync start bit to sampleNow
        4'b1000: if(sampleNow) RxD_state <= 4'b1001;  // bit 0
        4'b1001: if(sampleNow) RxD_state <= 4'b1010;  // bit 1
        4'b1010: if(sampleNow) RxD_state <= 4'b1011;  // bit 2
        4'b1011: if(sampleNow) RxD_state <= 4'b1100;  // bit 3
        4'b1100: if(sampleNow) RxD_state <= 4'b1101;  // bit 4
        4'b1101: if(sampleNow) RxD_state <= 4'b1110;  // bit 5
        4'b1110: if(sampleNow) RxD_state <= 4'b1111;  // bit 6
```

```
        4'b1111: if(sampleNow) RxD_state <= 4'b0010;  // bit 7
        4'b0010: if(sampleNow) RxD_state <= 4'b0000;  // stop bit
        default: RxD_state <= 4'b0000;
endcase

always @(posedge clk)
if(sampleNow && RxD_state[3]) RxD_data <= {RxD_bit, RxD_data[7:1]};

//reg RxD_data_error = 0;
always @(posedge clk)
begin
        RxD_data_ready <= (sampleNow && RxD_state==4'b0010 && RxD_bit);  // make sure a stop bit is received
        //RxD_data_error <= (sampleNow && RxD_state==4'b0010 && ~RxD_bit);  // error if a stop bit is not received
end

`ifdef SIMULATION
assign RxD_idle = 0;
`else
reg [l2o+1:0] GapCnt = 0;
always @(posedge clk) if (RxD_state!=0) GapCnt<=0; else if(OversamplingTick & ~GapCnt[log2(Oversampling)+1]) GapCnt <=
     GapCnt + 1'h1;
assign RxD_idle = GapCnt[l2o+1];
always @(posedge clk) RxD_endofpacket <= OversamplingTick & ~GapCnt[l2o+1] & &GapCnt[l2o:0];
`endif

endmodule


//////////////////////////////////////////////////////
// dummy module used to be able to raise an assertion in Verilog
module ASSERTION_ERROR();
endmodule


//////////////////////////////////////////////////////
module BaudTickGen(
        input clk, enable,
        output tick  // generate a tick at the specified baud rate * oversampling
);
parameter ClkFrequency = 50000000;
parameter Baud = 115200;
parameter Oversampling = 1;

function integer log2(input integer v); begin log2=0; while(v>>log2) log2=log2+1; end endfunction
localparam AccWidth = log2(ClkFrequency/Baud)+8;  // +/- 2% max timing error over a byte
reg [AccWidth:0] Acc = 0;
localparam ShiftLimiter = log2(Baud*Oversampling >> (31-AccWidth));  // this makes sure Inc calculation doesn't overflow
localparam Inc = ((Baud*Oversampling << (AccWidth-ShiftLimiter))+(ClkFrequency>>(ShiftLimiter+1)))/(ClkFrequency>>
     ShiftLimiter);
always @(posedge clk) if(enable) Acc <= Acc[AccWidth-1:0] + Inc[AccWidth:0]; else Acc <= Inc[AccWidth:0];
assign tick = Acc[AccWidth];
endmodule


//////////////////////////////////////////////////////
```